OS Orange 2

Unit 4

Vedant Sharma

PES2UG24CS577

4-J

17.04.2026

PES-VCS

Repository: https://github.com/Websurfingg21/PES2UG24CS577-pes-vcs

Phase 1

Screenshot 1A

```
vedant@Ubuntu24:~/Desktop/OS_Orange_2/PES2UG24CS577-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check
```

Screenshot 1B

```
vedant@Ubuntu24:~/Desktop/OS_Orange_2/PES2UG24CS577-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Phase 2

Screenshot 2A

```
vedant@Ubuntu24:~/Desktop/OS_Orange_2/PES2UG24CS577-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B

```
vedant@Ubuntu24:~/Desktop/OS_Orange_2/PES2UG24CS577-pes-vcs$ xxd .pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db
9ebb4 | head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a         blob 6.hello.
```

Phase 3

Screenshot 3A

```
vedant@Ubuntu24:~/Desktop/OS_Orange_2/PES2UG24CS577-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
vedant@Ubuntu24:~/Desktop/OS_Orange_2/PES2UG24CS577-pes-vcs$ ./pes add file1.txt file2.txt
vedant@Ubuntu24:~/Desktop/OS_Orange_2/PES2UG24CS577-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.h
  untracked:  commit.h
  untracked:  tree.h
  untracked:  Makefile
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  pes.c
  untracked:  object.c
  untracked:  index.c
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  commit.c
```

Screenshot 3B

```
vedant@Ubuntu24:~/Desktop/OS_Orange_2/PES2UG24CS577-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776519082 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776519090 6 file2.txt
```

Phase 4

Screenshot 4A

```
vedant@Ubuntu24:~/Desktop/OS_Orange_2/PES2UG24CS577-pes-vcs$ ./pes log
commit eed57b204c04ffdc95a357ace354831896e85cdd5b2e9186a4b4c1843177b8c4
Author: PES User <pes@localhost>
Date:   1776534436

    Add farewell

commit 56e109dc7ec52e197109e222d4bf0688da6709fc059dacb6b595ed6c9958778b
Author: PES User <pes@localhost>
Date:   1776534417

    Add worldell

commit 8f839575c49006c559741905be37cd2944146b204a670ca4b104cbcd0f5cb3fa
Author: PES User <pes@localhost>
Date:   1776534394

    Initial committial commiQ
```

Screenshot 4B

```
vedant@Ubuntu24:~/Desktop/OS_Orange_2/PES2UG24CS577-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/56/e109dc7ec52e197109e222d4bf0688da6709fc059dacb6b595ed6c9958778b
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/8f/839575c49006c559741905be37cd2944146b204a670ca4b104cbcd0f5cb3fa
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ee/d57b204c04ffdc95a357ace354831896e85cdd5b2e9186a4b4c1843177b8c4
.pes/refs/heads/main
```

Screenshot 4C

```
vedant@Ubuntu24:~/Desktop/OS_Orange_2/PES2UG24CS577-pes-vcs$ cat .pes/refs/heads/main
eed57b204c04ffdc95a357ace354831896e85cdd5b2e9186a4b4c1843177b8c4
vedant@Ubuntu24:~/Desktop/OS_Orange_2/PES2UG24CS577-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Phase 5

Q5.1

pes checkout <branch> works by first updating the .pes/HEAD file to point to the selected branch (e.g., ref: refs/heads/<branch>), where each branch is just a file containing a commit hash. The system then reads that commit, retrieves its associated tree object, and reconstructs the working directory by recursively writing all files and directories stored in that tree. This includes creating directories, writing file contents from blob objects, and removing any files that are not present in the target commit. The index should also be updated to reflect the checked-out state. The operation is complex because it must correctly handle nested directory structures, file–directory conflicts, deletion of extra files, and potential overwriting of uncommitted changes, all while maintaining consistency of the repository.

Q5.2

To detect a "dirty working directory" conflict, we first compare the current working directory against the index, and then compare the index against the target branch's tree using the object store. First, for each tracked file in the index, check if it has been modified in the working directory by comparing metadata or recomputing its hash and comparing it with the stored blob hash in the index. If it differs, the file is "dirty". Next, load the target branch's commit, parse its tree, and determine the blob hash for that same file in the target branch. If the file exists in both the index and the target tree but their hashes differ, then switching branches would overwrite the user's uncommitted changes. In that case, checkout must refuse. This detection uses only the index, the working directory, and the object store.

Q5.3

In a detached HEAD state, HEAD points directly to a specific commit hash instead of a branch, so any new commits you make are not attached to any branch. These commits still get created and stored in the object store, and HEAD moves forward to them, but no branch reference is updated to remember them. As a result, once you switch away to another branch, there is no reference pointing to those commits, so they become dangling and may eventually be garbage collected. To recover them, the user must create a branch (or tag) pointing to the current commit before leaving the detached state, or use tools like log to find the commit hash and manually create a branch reference to it.

Phase 6

Q6.1

To remove unreachable objects, we perform a traversal starting from all branch heads and HEAD. From each root commit, we recursively visit all reachable commits, trees, and blobs, marking their hashes in a hash set for efficient lookup. After this mark phase, we scan the entire object store and delete any object whose hash is not in the reachable set. A hash set is used because it provides constant-time membership checks and avoids revisiting objects. For a repository with 100,000 commits and 50 branches, the number of visited objects is approximately the number of unique reachable objects (around 100,000 commits plus associated trees and blobs), not 50×100,000, since most branches share history.

Q6.2

Running garbage collection (GC) concurrently with a commit is dangerous because GC may delete objects that are temporarily unreachable but about to become reachable. A race condition can occur as follows: a commit operation first writes new objects (blobs → trees → commit object) into the object store, but before it updates any branch reference (like .pes/refs/heads/master) or HEAD, those objects are not yet reachable from any root. If GC runs at this exact moment, it scans from existing branch heads, does not see the newly created objects, and marks them as unreachable. It may then delete them. Immediately after, the commit operation updates the branch to point to the new commit, but now that commit (or its tree/blob objects) has already been deleted, leaving the repository in a corrupted state.

Git avoids this using multiple safeguards. First, objects are written atomically and are protected by file locks during updates to refs, preventing GC from running simultaneously with critical operations. Second, Git uses a grace period (based on file modification time), meaning very recent objects are not deleted even if currently unreachable. Third, Git maintains reflogs, which act as additional roots, ensuring recently referenced commits are still considered reachable. Together, these mechanisms prevent GC from deleting objects that are in the process of being committed.